

Detección y segmentación 3D de metástasis cerebrales en RM

IA para RM cerebral: encontrar y perfilar metástasis.
Pensada para ser sensible, estable y transparente.

Índice de contenidos

Índice de contenidos	1
1. Introducción	3
2. Descripción del Dataset	4
2.1 Modalidades de imagen	4
2.2 Máscaras de segmentación.....	4
2.3 Espacios de referencia: SRI24 vs UCSD	5
2.4 Distribución de clases y desbalance.....	5
2.5 Formato de los datos	5
3. Preprocesamiento de Datos	6
3.1 Corrección de bias field (N4ITK)	6
3.2 Normalización por canal (z-score).....	6
3.3 Redimensionado a 128^3	6
3.4 Generación de parches 3D	7
3.5 Estrategias de oversampling	7
3.6 División train/validation por paciente	7
4. Arquitectura y Modelos Propuestos	8
4.1 Modelo Detector Binario	8
4.2 Modelos de Segmentación Multiclase	8
4.2.1 Attention U-Net 3D con Deep Supervision.....	8
4.2.2 U-Net++ 3D con Deep Supervision.....	9
4.2.3 Modelos experimentales (EfficientNet-like)	9
4.3 Funciones de pérdida	9
4.4 Estrategias de Ensemble.....	9
5. Entrenamiento y Métricas	10
5.1 Configuración del entrenamiento	10
5.2 Callbacks utilizados.....	10
5.3 Estrategia híbrida de parches	10
5.4 Funciones de pérdida	11
6. Resultados y Evaluación	12
6.1 Resultados del modelo detector binario.....	12
6.2 Resultados de los modelos de segmentación	12

6.3 Estrategias de ensemble	14
6.4 Visualización de casos representativos	14
6.5 Volumetría de tumores	15
7. Discusión.....	15
7.1 Fortalezas del enfoque	15
7.2 Limitaciones identificadas	15
7.3 Comparación con la literatura	16
7.4 Lecciones aprendidas	16
7.5 Productivizar el sistema	16
7.6 Retos técnicos y soluciones.....	17
8. Conclusiones y Futuras Mejoras.....	18
8.1 Conclusiones principales	18
8.2 Limitaciones	19
8.3 Futuras mejoras	19
9. Referencias.....	20
ANEXOS	22
Anexo A — Sección 1 del html: Preprocesado de datos.	22
Anexo B — Sección 3 del html: Creación de Patches.....	24
Anexo C — Sección 4 del html: Modelo Detector Binario (etapa 1)	26
Anexo D — Sección 5 del html: Segmentadores 3D (etapa 2)	28
Anexo E — Sección 5 del html: Entrenamiento y pérdidas.....	31
Anexo F — Sección 6 del html: Ensemble voxel-wise por clase (con TTA).....	33
Anexo G — Sección 6.5 del html: Evaluación y métricas	35
Anexo H — Sección 7 del html: Estudio de casos.....	37
Anexo I — Sección 8 del html: Productivización en Streamlit.....	42
Anexo L —Acceso a los datos y modelos entrenados.	46

1. Introducción

Las metástasis cerebrales representan el tumor intracraneal más frecuente en adultos, superando en incidencia a los tumores primarios del sistema nervioso central (SNC). Su diagnóstico temprano y seguimiento resultan fundamentales tanto para la planificación terapéutica como para el pronóstico de los pacientes.

La amplia variabilidad y complejidad en la morfología de las metástasis cerebrales, así como sus diversas localizaciones, convierte su detección y segmentación en una tarea compleja para los especialistas en radiología. Esto se traduce en una alta especialización y dedicación para los profesionales del campo de la neurorradiología, normalmente únicamente presentes en hospitales de tercer nivel. La prueba de imagen de elección para la detección de metástasis cerebrales es la Resonancia Magnética (RM) que ha demostrado un rendimiento superior a la de la tomografía computarizada (TC).

En los últimos años, los modelos de aprendizaje profundo (Deep Learning), y en particular las redes neuronales convolucionales 3D, han demostrado un alto potencial para automatizar este tipo de tareas en imagen médica. Un referente en este ámbito es el Brain Tumor Segmentation Challenge (BraTS), un reto internacional que impulsa el desarrollo de técnicas avanzadas de segmentación de tumores cerebrales y que se ha consolidado como benchmark de investigación en este campo.

Este Trabajo Fin de Máster se enmarca en la tarea 4 del desafío BraTS 2025, orientada a la detección y segmentación de metástasis cerebrales en imágenes de RM. El objetivo general consiste en diseñar un sistema en dos etapas capaz de:

- Detectar regiones sospechosas de metástasis mediante un clasificador binario de parches.
- Segmentar con precisión las lesiones identificadas, diferenciando entre los distintos componentes tumorales (lesión tumoral, necrosis y edema peritumoral).

Para alcanzar este objetivo, se exploran diversas arquitecturas de redes neuronales 3D (U-Net, Attention U-Net, U-Net++, modelos tipo EfficientNet), combinadas con estrategias de entrenamiento basadas en parches tridimensionales y técnicas de oversampling para abordar el desbalance de clases. Los resultados se evalúan utilizando métricas estándar como el coeficiente Dice por clase, y se estudian diferentes estrategias de ensemble dirigidas a mejorar el rendimiento global y la segmentación de subgrupos específicos.

Finalmente, este informe se organiza de la siguiente manera: la sección 2 describe el conjunto de datos empleado; la sección 3 detalla el preprocesamiento aplicado; en la sección 4 se presentan las arquitecturas propuestas; las secciones 5 y 6 recogen el proceso de entrenamiento y los resultados obtenidos; y, por último, las secciones 7 y 8 incluyen la discusión, las conclusiones principales y las posibles mejoras futuras.

2. Descripción del Dataset

El presente trabajo utiliza datos del Brain Tumor Segmentation Challenge 2025 (BraTS 2025), concretamente de la Task 4: Segmentación de Metástasis Cerebrales. Este reto, organizado en el marco del Medical Image Computing and Computer Assisted Intervention (MICCAI), constituye una de las competiciones internacionales de referencia en el ámbito de la inteligencia artificial aplicada a la imagen médica.

La tarea 4 (BraTS-METS 2025) se centra en la detección y segmentación automática de metástasis cerebrales en RM multimodales. El objetivo es identificar de manera precisa la localización y el volumen de las lesiones metastásicas, diferenciando entre sus distintos componentes internos. Este tipo de análisis es fundamental en el ámbito clínico, ya que:

- Facilita el diagnóstico y la detección temprana de metástasis y por tanto, una mejoría del pronóstico de los pacientes.
- Contribuye a la planificación terapéutica en el campo de la radioterapia.
- Facilita el seguimiento de los tumores durante y después del tratamiento.

2.1 Modalidades de imagen

El dataset está compuesto por estudios de RM multimodal, cada uno con cuatro modalidades distintas:

- **T1-nativa (T1n):** imágenes potenciadas en T1 que proporcionan una información anatómica básica del cerebro.
- **T2 (T2w):** imágenes potenciadas en T2. Permite visualizar la presencia de agua y complementa la visualización de las imágenes potenciadas en T1.
- **T2-FLAIR (T2f):** "Fluid-Attenuated Inversion Recovery". Esta técnica de RM suprime la señal del líquido cefalorraquídeo para facilitar de visualización del edema peritumoral (la extravasación y acumulación extracelular de líquido en el parénquima cerebral).
- **T1 con contraste (T1c):** la captación de contraste permite identificar el tejido tumoral donde se ha producido una rotura de la barrera hematoencefálica (el contraste administrado por vía intravenosa sale de los vasos sanguíneos y se acumula en la zona tumoral, originando un "realce" que lo hace visible en las pruebas de imagen).

La combinación de estas cuatro secuencias ofrece una visión global de la anatomía y morfología de las estructuras cerebrales tanto normales como patológicas, lo que resulta esencial para que los modelos de Deep Learning puedan aprender patrones robustos.

2.2 Máscaras de segmentación

Cada caso incluye una máscara manual de referencia (ground truth), generada por expertos en neuroimagen, que identifica las distintas regiones tumorales. La codificación de las clases es la siguiente:

- **Clase 0:** Fondo (tejido sano).
- **Clase 1:** NETC: Núcleo tumoral no realzado.

- **Clase 2:** SNFH: Edema peritumoral (Swelling / Non-Enhancing FLAIR Hyperintensity).
- **Clase 3:** ET: Tumor realzado (Enhancing Tumor).
- **Clase 4:** RC: Cavity de resección (Resection Cavity).

Estas clases permiten no solo localizar las lesiones, sino también distinguir entre diferentes componentes clínicamente relevantes para la planificación del tratamiento.

2.3 Espacios de referencia: SRI24 vs UCSD

El conjunto de datos de BraTS 2025 se encuentra disponible en dos espacios de referencia:

- **SRI24:** volúmenes ya registrados a un espacio común de referencia anatómica, lo que facilita el entrenamiento de modelos al estandarizar la localización espacial.
- **UCSD** (espacio nativo): volúmenes en su espacio original de adquisición, sin normalización espacial previa. Este formato refleja la variabilidad clínica real, aunque incrementa la dificultad de la segmentación automática.

En este trabajo se han empleado ambos espacios: el SRI24 como base más homogénea y el espacio nativo UCSD para evaluar la robustez del pipeline frente a datos más cercanos a la práctica clínica.

2.4 Distribución de clases y desbalance

Uno de los principales retos del dataset es el fuerte desbalance entre clases. En particular:

- La clase 4 (RC) aparece únicamente en ~12.6% de los casos.
- Las clases 1 (NETC), 2 (SNFH) y 3 (ET) presentan mayor frecuencia, aunque también con diferencias notables entre ellas.
- El fondo (clase 0) ocupa la mayoría de voxels en cada volumen, lo que genera un fuerte desequilibrio frente a las regiones tumorales.

Este desbalance motiva el uso de estrategias de oversampling de parches y de funciones de pérdida adaptadas (como la combinación de Dice Loss y Focal Loss) para mejorar la capacidad del modelo en las clases menos representadas.

Se puede encontrar la distribución de clases en el Anexo B.

2.5 Formato de los datos

Los datos se encuentran en formato NIfTI (.nii.gz), ampliamente utilizado en imagen médica. Cada paciente está identificado por un ID único (por ejemplo, BraTS-MET-01071-001), y se proporcionan tanto las imágenes multimodales como las máscaras de segmentación correspondientes.

Para facilitar el manejo en el pipeline de entrenamiento, los volúmenes fueron preprocesados y posteriormente convertidos a matrices NumPy (.npy) y archivos

comprimidos (.npz), lo que permitió trabajar con parches 3D de 64^3 y 96^3 de manera eficiente durante la fase de modelización.

3. Preprocesamiento de Datos

El preprocesamiento constituye una etapa fundamental en cualquier pipeline de aprendizaje profundo aplicado a imágenes médicas. En el caso de BraTS 2025, los volúmenes originales presentan diferentes intensidades, artefactos de adquisición y dimensiones variables, lo que hace necesario aplicar una serie de transformaciones previas antes de entrenar los modelos.

A continuación, se detallan los principales pasos realizados:

3.1 Corrección de bias field (N4ITK)

Las imágenes de RM suelen presentar inhomogeneidades de intensidad debidas a artefactos del campo magnético, lo que puede dificultar la diferenciación entre tejidos. Para mitigar este efecto, se aplicó la corrección de bias field mediante el algoritmo N4ITK.

Este proceso ajusta las intensidades de cada volumen MRI de forma que se reducen gradientes no deseados y se mejora la homogeneidad dentro de cada tejido, facilitando tanto la normalización posterior como la segmentación automática.

3.2 Normalización por canal (z-score)

Cada modalidad de resonancia presenta rangos de intensidad distintos. Para estandarizar las entradas al modelo, se aplicó una normalización por canal utilizando el método z-score, definida como:

$$I_{\text{norm}} = \frac{I - \mu}{\sigma}$$

donde μ y σ corresponden a la media y desviación estándar de los voxels no nulos de cada modalidad.

Con ello, se garantiza que todas las entradas presenten una distribución con media cero y varianza unitaria, mejorando la estabilidad del entrenamiento y evitando que ciertas modalidades dominen sobre otras.

3.3 Redimensionado a 128^3

Los volúmenes originales del dataset presentan dimensiones variables según el caso y el hospital de adquisición. Para unificar los datos y adaptarlos a las arquitecturas convolucionales 3D empleadas, cada volumen fue redimensionado a un tamaño estándar de $128 \times 128 \times 128$ voxels.

Este tamaño constituye un compromiso entre la resolución espacial suficiente para capturar lesiones pequeñas y la viabilidad computacional para entrenar modelos en GPU.

3.4 Generación de parches 3D

Dado que el entrenamiento directo con volúmenes completos puede resultar muy costoso en memoria, se optó por una estrategia de entrenamiento basado en parches 3D.

- Se extrajeron parches cúbicos de 64^3 y 96^3 voxeles, que permiten entrenar modelos con diferentes resoluciones.
- Los parches de 64^3 se emplearon principalmente en arquitecturas como Attention U-Net 3D, mientras que los de 96^3 se usaron para modelos más complejos como U-Net++ 3D con deep supervision.

Este enfoque no solo reduce la carga computacional, sino que además aumenta el número de muestras de entrenamiento disponibles.

Puede encontrarse un ejemplo de parches en el Anexo B y sección 3 del html.

3.5 Estrategias de oversampling

Uno de los principales retos del dataset es el desbalance de clases, especialmente en la clase 4 (cavidad de resección), presente únicamente en ~12.6% de los casos. Para mitigar este problema, se aplicaron las siguientes estrategias de oversampling:

- Mayor proporción de parches que contienen clases minoritarias (1 y 4): se generaron más parches a partir de regiones donde estas clases estaban presentes.
- Parches negativos (fondo puro): se incluyó un porcentaje controlado de parches sin voxeles tumorales para reforzar la capacidad del modelo de diferenciar entre tejido sano y lesiones.

Con ello, se consiguió un dataset de entrenamiento más equilibrado, favoreciendo la sensibilidad en clases minoritarias sin perder robustez en el reconocimiento de tejido sano.

3.6 División train/validation por paciente

Para evitar fugas de información (data leakage), la división entre conjuntos de entrenamiento y validación se realizó a nivel de paciente completo.

De esta manera, todos los parches pertenecientes a un mismo paciente se asignan exclusivamente a train o a validation, evitando que el modelo “vea” información del mismo caso en ambas fases.

La proporción final empleada fue aproximadamente 80% de pacientes para entrenamiento y 20% para validación, garantizando representatividad en ambas particiones.

4. Arquitectura y Modelos Propuestos

El sistema desarrollado en este Trabajo Fin de Máster se basa en un pipeline en dos etapas:

1. Detección de regiones candidatas mediante un modelo binario de clasificación de parches.
2. Segmentación multiclase detallada sobre las regiones detectadas, utilizando arquitecturas avanzadas de redes convolucionales 3D.

Este enfoque en cascada responde a la necesidad de reducir la complejidad espacial del problema (ya que la mayoría de voxels corresponden a fondo sano) y mejorar la sensibilidad en la detección de lesiones pequeñas.

4.1 Modelo Detector Binario

La primera etapa del pipeline consiste en un clasificador de parches 3D encargado de identificar si un parche contiene o no metástasis.

- **Entrada:** parches de 64^3 voxels con las cuatro modalidades de resonancia (T1n, T1c, T2f, T2w).
- **Arquitectura:** CNN 3D con bloques convolucionales, normalización por lotes y activaciones LeakyReLU. Se incluyeron conexiones residuales para mejorar la estabilidad del entrenamiento.
- **Salida:** probabilidad binaria (parche positivo o negativo).
- **Objetivo:** maximizar la sensibilidad (recall) para garantizar que no se descarten parches con lesiones.

Este modelo actúa como filtro inicial, reduciendo la cantidad de parches procesados por el segmentador y concentrando el análisis en regiones relevantes.

Puede encontrarse más información en el Anexo C y sección 4 del html.

4.2 Modelos de Segmentación Multiclase

En la segunda etapa se desarrollaron distintos modelos de segmentación voxel a voxel capaces de distinguir entre los componentes tumorales (clases 1 a 4). Se desarrolla brevemente en el Anexo D y de forma completa en la sección 5 del html.

4.2.1 Attention U-Net 3D con Deep Supervision

- **Base:** arquitectura U-Net 3D clásica, con encoder-decoder y skip connections.
- **Mecanismos de atención:** se añadieron attention gates en las conexiones de salto, que permiten al modelo focalizarse en regiones relevantes.
- **Deep supervision:** se incorporaron salidas intermedias en distintas escalas, que se combinan durante el entrenamiento para facilitar la convergencia.

- **Entrada:** parches de 64^3 .
- **Ventaja principal:** buen desempeño en lesiones pequeñas, especialmente en la clase 4 (cavidad de resección) aunque se observó un error con este modelo y por tanto no se llevó a producción.

4.2.2 U-Net++ 3D con Deep Supervision

- **Arquitectura** más densa y jerárquica: basada en múltiples caminos de conexión entre encoder y decoder, lo que enriquece la representación multiescala.
- **Deep supervision:** salidas auxiliares para estabilizar el entrenamiento y mejorar la segmentación en distintas resoluciones.
- **Entrada:** parches de 96^3 , que aportan mayor contexto espacial.
- **Ventaja principal:** Buen rendimiento en las clases 1, 2 y 3 donde la estructura tumoral es más extensa y heterogénea. Dio buenos resultados en la clase 4 con parches de 64.

4.2.3 Modelos experimentales (EfficientNet-like)

Además de las arquitecturas principales, se exploró una variante inspirada en EfficientNet 3D combinada con un bloque ASPP (Atrous Spatial Pyramid Pooling) en el cuello del modelo. Aunque experimental, permitió contrastar los resultados con enfoques más recientes de segmentación.

4.3 Funciones de pérdida

Dado el fuerte desbalance de clases, se optó por una función de pérdida combinada:

$$L = \alpha L_{Dice} + \beta L_{Focal}$$

- **Dice Loss:** favorece la superposición entre predicción y máscara real, útil en clases poco representadas.
- **Focal Loss** ($\gamma=1.5$, α personalizado): penaliza con mayor fuerza los errores en las clases minoritarias, como la clase 4.
- **Configuración final:** $\alpha = (0.2, 1, 1, 1, 3)$ para dar mayor peso a la clase 4.

4.4 Estrategias de Ensemble

Para aprovechar los puntos fuertes de cada modelo, se evaluaron distintas estrategias de combinación:

- Promedio simple (avg ensemble): promedio de las probabilidades softmax de los modelos.
- Voxel-wise softmax ensemble: combinación voxel a voxel, que demostró una mejora clara en las clases minoritarias.
- Oracle ensemble (benchmark teórico): selección del mejor resultado por caso y clase, usado como referencia para estimar el límite superior alcanzable.

El ensemble final adoptado priorizó el uso de U-Net++ 96³ para clases 1–3 y U-Net ++ 3D 64³ para la clase 4, lo que permitió optimizar el rendimiento global y a la vez mejorar la detección de metástasis pequeñas.

5. Entrenamiento y Métricas

El entrenamiento de los modelos se diseñó teniendo en cuenta la complejidad del problema (segmentación multiclase con fuerte desbalance de clases) y las limitaciones computacionales asociadas al trabajo con volúmenes 3D.

5.1 Configuración del entrenamiento

- **Entorno de cómputo:** los experimentos se llevaron a cabo en un entorno con GPU NVIDIA RTX A6000, lo que permitió entrenar modelos 3D con parches de hasta 96³ voxeles.
- **Precisión mixta:** se utilizó la política mixed_float16 para reducir el consumo de memoria y acelerar el entrenamiento.
- **Batch size:** 8 para parches 64³ y 96³, ajustado al límite de memoria de la GPU.
- **Optimización:** se empleó el optimizador Adam con una tasa de aprendizaje inicial de 10^{-4}
- **Planificación de LR:**
 - WarmupLR: incremento gradual de la tasa de aprendizaje durante las primeras épocas.
 - ReduceLROnPlateau: reducción automática de la tasa de aprendizaje al detectar estancamiento en la validación.
- **Supervisión profunda con annealing** (apagado progresivo): Para estabilizar el entrenamiento de U-Net++ con deep supervision, se combina la pérdida de la salida principal con las salidas auxiliares y se decrecen sus pesos durante el entrenamiento (*annealing*), de modo que guían al inicio y no penalizan al final.

5.2 Callbacks utilizados

Para garantizar estabilidad y reproducibilidad, se incorporaron múltiples callbacks:

- **EarlyStopping:** finalización anticipada si no se observaba mejora en la pérdida de validación tras 13 épocas.
- **ModelCheckpoint:** guardado del mejor modelo según la pérdida de validación.
- **CSVLogger y TensorBoard:** registro detallado del proceso de entrenamiento y métricas.
- **BackupAndRestore:** protección frente a interrupciones en la sesión de entrenamiento.

5.3 Estrategia híbrida de parches

Se aplicó una estrategia híbrida entrenando con diferentes resoluciones de parches:

- **Parches de 64³**: permitieron focalizarse en detalles locales y entrenar modelos más ligeros.
- **Parches de 96³**: aportaron mayor contexto espacial, mejorando la segmentación en clases más extensas (1, 2 y 3).

Esta combinación permitió comparar arquitecturas y explorar el efecto de la resolución en el rendimiento.

5.4 Funciones de pérdida

Los modelos fueron entrenados con la función de pérdida combinada descrita en la sección anterior:

- **Dice Loss**: asegura que las predicciones se ajusten a la forma y volumen de las regiones tumorales.
- **Focal Loss** ($\gamma=1.5$): se utilizó para dar mayor peso a los errores en clases minoritarias.
- **Pesos α** : configurados como (0.2, 1, 1, 1, 3), con el fin de aumentar la sensibilidad en la clase 4.

5.5 Métricas de evaluación

El desempeño de los modelos se evaluó utilizando métricas estándar en segmentación médica, tanto a nivel de parche como de volumen reconstruido:

- **Coefficiente Dice por clase (1–4)**: mide la superposición entre la segmentación predicha y la máscara real. Es la métrica principal del reto BraTS y refleja qué tan bien se ha segmentado cada tipo de metástasis.
- **Mediana de Dice por clase**: se utilizó la mediana (en lugar de la media) para representar de forma más robusta el rendimiento global, al ser menos sensible a casos extremos.
- **Normalized Surface Dice (NSD)**: métrica basada en la coincidencia de las superficies segmentadas, considerando un umbral de tolerancia (por ejemplo, 3 mm). Evalúa la precisión en los bordes, un aspecto clínicamente relevante.
- **Hausdorff Distance 95 (HD95)**: mide la distancia máxima (percentil 95) entre los bordes reales y predichos. Se reportó la media de HD95 por clase, como indicador complementario a la NSD para evaluar la precisión superficial.
- **Recall (sensibilidad) en el detector binario**: usada para evaluar la primera etapa del pipeline (detección), asegurando la localización de la mayoría de parches tumorales.
- **AUC-ROC del detector**: métrica complementaria que resume la capacidad del modelo binario para distinguir entre parches positivos y negativos a distintos umbrales.

Estas métricas se calcularon tanto en el conjunto de validación como en el test oficial, y permiten analizar el rendimiento del pipeline completo en condiciones más cercanas al uso clínico.

6. Resultados y Evaluación

En esta sección se presentan los resultados obtenidos tras el entrenamiento de los modelos, tanto en la fase de detección binaria de parches como en la fase de segmentación multiclase. Asimismo, se analizan distintas estrategias de ensemble y se incluyen ejemplos visuales en casos representativos del conjunto de validación.

6.1 Resultados del modelo detector binario

El modelo detector binario, basado en una CNN 3D, alcanzó un rendimiento robusto en la clasificación de parches:

- **AUC-ROC:** valores superiores a 0.90 en validación.
- **Recall:** priorizado durante la optimización, superando el 95%, garantizando que la mayoría de parches con metástasis fueran detectados.
- **Precisión:** más moderada debido a la estrategia de oversampling y al sesgo hacia la sensibilidad, pero suficiente dado el rol de filtro inicial.

Estos resultados confirman la utilidad del detector como primera etapa del pipeline, reduciendo drásticamente el número de parches que pasan al segmentador sin comprometer la detección de lesiones.

6.2 Resultados de los modelos de segmentación

En la segunda etapa se evaluaron distintas arquitecturas de segmentación multiclase:

El modelo Unet ++ 3D a 64³ muestra un rendimiento sólido en todas las clases y un salto claro en RC (clase 4), que es la más difícil.

Detalle por clase.

- **C1- NETC:** ligera caída de Dice (-0.0086). Es coherente con parches pequeños: el 64³ prioriza detalle local y puede perder algo de contexto del núcleo no realzante.
- **C2 - SNFH:** mejora +0.0221 en Dice; el TTA estabiliza bordes del edema/infiltración, que suelen ser difusos.
- **C3 - ET:** se mantiene estable (± 0.0000), lo cual indica que ya estaba bien resuelto y TTA no introduce ruido.
- **C4 - RC:** mejora marcada +0.0480 en Dice; el 64³ capta mejor las cavidades pequeñas e irregulares y TTA reduce la varianza de predicción.

En U-Net++ 64³, el NSD también mejora de forma consistente, con los mayores incrementos en RC (clase 4) y SNFH (clase 2). Aunque el Dice en 64³ puede moverse poco en NETC/ET, el NSD confirma bordes mejor colocados tras TTA, especialmente en lesiones pequeñas o de contorno irregular. En ET (clase 3) el NSD se mantiene/crece ligeramente, lo que indica estabilidad.

Conclusión 64³. TTA aporta robustez especialmente en clases de lesión pequeña o borde complejo (C2 y C4), manteniendo ET estable.

Clase	Dice (sin TTA)	Dice (con TTA)	Mejora
1 – NETC	0.7586	0.7500	-0.0086
2 – SNFH	0.7557	0.7778	+0.0221
3 – ET	0.7610	0.7610	±0.0000
4 – RC	0.6187	0.6667	+0.0480

Clase	Dice (sin TTA)	Dice (con TTA)	Mejora
1 – NETC	0.7586	0.7500	-0.0086
2 – SNFH	0.7557	0.7778	+0.0221
3 – ET	0.7610	0.7610	±0.0000
4 – RC	0.6187	0.6667	+0.0480

El modelo Unet ++ 3D a 96³ aporta más contexto y rinde mejor en clases 1–3 de media; con TTA se observan:

- **C1 - NETC y C2 - SNFH:** leve descenso de Dice (-0.0500; -0.0232). Con más contexto, el modelo puede “suavizar” límites internos y penalizar Dice volumétrico en casos con gran variabilidad.
- **C3 - ET:** mejora +0.0110 en Dice, consistente con bordes más nítidos en realce.
- **C4 - RC:** pasa de 0.0000 → 0.2000 (mejora significativa); indica que TTA ayuda a recuperar casos donde el modelo sin TTA fallaba por completo.

Clase	Dice (sin TTA)	Dice (con TTA)	Mejora
1 – NETC	0.8000	0.7500	-0.0500
2 – SNFH	0.8010	0.7778	-0.0232
3 – ET	0.7500	0.7610	+0.0110
4 – RC	0.0000	0.2000	↑ significativa

Clase	NSD (sin TTA)	NSD (con TTA)	Mejora
1 – NETC	0.8333	0.8424	+0.0091
2 – SNFH	0.8836	0.8841	+0.0005
3 – ET	0.9091	0.9221	+0.0130
4 – RC	0.4906	0.5490	+0.0584

El NSD sube en todas las clases (C1: +0.0091; C2: +0.0005; C3: +0.0130; C4: +0.0584), lo que significa mejor coincidencia de superficies incluso donde el Dice cayó ligeramente. Es decir, los bordes están mejor colocados, aunque el volumen cambie poco.

En general, las métricas globales confirmaron la dificultad del problema en las clases minoritarias, especialmente la clase 4, mientras que el desempeño en las clases 1–3 fue notablemente más alto.

Puede encontrarse la evolución del Train y Val loss así como los dice en el anexo D.

La evaluación de los distintos modelos se encuentra en la sección 6 del html.

6.3 Estrategias de ensemble

Con el fin de maximizar el rendimiento, se exploraron distintas combinaciones de modelos:

- Promedio simple (avg ensemble): combinación directa de las probabilidades softmax de los modelos.
- Voxel-wise softmax ensemble: demostró mejoras claras en las clases 2, 3 y 4 respecto al promedio, elevando el Dice en clase 3 de 0.55 a 0.64 y manteniendo una mediana de 1.0 en clase 4.
- Oracle ensemble (benchmark teórico): seleccionando, para cada caso y clase, el mejor resultado entre modelos. Si bien no es viable en producción, sirvió como referencia del techo máximo de rendimiento.

En la práctica, se adoptó un ensemble híbrido que combinó U-Net++ 96³ para las clases 1–3 y U-Net++ 3D 64³ para la clase 4, equilibrando rendimiento global y sensibilidad en lesiones pequeñas.

Para ver el desarrollo del ensemble Anexo F, sección 6 del html.

6.4 Visualización de casos representativos

Para evaluar la calidad de las segmentaciones, se analizaron varios casos del conjunto de validación:

- **BraTS-MET-01071-001**: caso con todas las clases presentes, que permitió validar el rendimiento equilibrado de los modelos.
- **BraTS-MET-01087-001**: caso con muy buenos Dice en clases 1, 2 y 3, representativo del potencial del modelo U-Net++.
- **BraTS-MET-00286-000**: caso con resultados deficientes en clases 1–3, útil para el análisis de errores y limitaciones del pipeline.

Las segmentaciones se visualizaron superpuestas sobre la modalidad T1c, mostrando la coherencia espacial de las predicciones respecto a las regiones reales.

Pueden encontrarse los resultados de las máscaras predichas en el Anexo I (Sección 7 del html). **GT** es la máscara real y **predicción** la hecha por el modelo.

6.5 Volumetría de tumores

Además de las métricas de segmentación, se calcularon los volúmenes de cada clase tumoral en mm^3 , a partir de las máscaras reconstruidas en formato NIfTI. Esta información es clínicamente relevante, ya que aporta un indicador cuantitativo del tamaño tumoral y su evolución potencial en seguimientos longitudinales.

7. Discusión

Los resultados obtenidos ponen de manifiesto tanto las fortalezas como las limitaciones del pipeline propuesto para la detección y segmentación de metástasis cerebrales en el reto **BraTS 2025 (Task 4)**.

7.1 Fortalezas del enfoque

- **Pipeline en cascada:** la división del problema en dos etapas (detección binaria y segmentación multiclase) permitió reducir drásticamente el espacio de búsqueda y mejorar la sensibilidad global. El detector binario alcanzó valores de recall superiores al 95%, garantizando que la mayoría de parches con lesiones pasaran a la segunda etapa.
- **Arquitecturas complementarias:** el uso combinado de U-Net++ 3D (96³) y U-Net ++ 3D (64³) mostró que distintos modelos pueden especializarse en clases diferentes: el primero en estructuras tumorales más extensas (clases 1–3) y el segundo en lesiones pequeñas (clase 4).
- **Funciones de pérdida adaptadas:** la combinación de Dice Loss y Focal Loss con ponderación por clase permitió mejorar la representación de las clases minoritarias, especialmente en el caso de la cavidad de resección.
- **Estrategias de ensemble:** el ensemble voxel-wise mejoró de forma consistente el rendimiento en las clases 2, 3 y 4, confirmando que la combinación inteligente de modelos supera a las arquitecturas individuales.

7.2 Limitaciones identificadas

- **Clase 4 (RC):** pese a las mejoras introducidas, esta clase continuó mostrando baja performance (Dice ≈ 0.36 en validación) debido a su escasa representación ($\sim 12.6\%$ de los casos) y a la alta variabilidad morfológica de las cavidades de resección.
- **Falsos negativos** en lesiones pequeñas: algunos casos del conjunto de validación mostraron fallos en la detección de metástasis de reducido tamaño, lo que evidencia la necesidad de modelos con mayor sensibilidad en estas situaciones.
- **Variabilidad entre espacios** (SRI24 vs UCSD): los modelos entrenados en el espacio normalizado (SRI24) presentaron pérdidas de rendimiento al evaluar en UCSD, reflejando la dificultad de generalizar a contextos clínicos no estandarizados.

- **Desbalance de clases:** aunque mitigado con oversampling y pérdidas adaptadas, el desbalance sigue siendo un factor crítico que limita la estabilidad del entrenamiento y el rendimiento final.

7.3 Comparación con la literatura

- Estudios previos en BraTS (ediciones 2023 y 2024) ya habían destacado la dificultad de la clase 4, reportando métricas significativamente más bajas frente a otras clases.
- La estrategia en cascada detector → segmentador coincide con enfoques propuestos en literatura reciente, que remarcan la utilidad de filtrar primero regiones candidatas para aumentar la eficiencia computacional y la sensibilidad.
- El uso de ensembles se alinea con la práctica común en competencias BraTS, donde los equipos punteros integran múltiples arquitecturas para maximizar resultados.

7.4 Lecciones aprendidas

- La heterogeneidad de las metástasis cerebrales requiere enfoques flexibles, donde distintos modelos se especialicen en diferentes tipos de lesiones.
- El desbalance extremo de clases no puede solucionarse únicamente con oversampling, sino que exige funciones de pérdida y arquitecturas adaptadas.
- La validación en espacios nativos (UCSD) es clave para garantizar la aplicabilidad clínica del sistema, ya que un pipeline entrenado solo en espacios normalizados puede no generalizar correctamente en escenarios reales.

7.5 Productivizar el sistema

Como fase final del proyecto, se desarrolló una aplicación interactiva que permite ejecutar el pipeline completo sobre nuevas resonancias, facilitando su uso en entornos no técnicos. Esta herramienta fue implementada en Streamlit, ejecutándose en un entorno con GPU (Paperspace) y permite cargar tanto volúmenes preprocesados (.npy) como resonancias originales (.nii.gz, 4 modalidades).

La app automatiza todo el flujo de trabajo, incluyendo:

- Preprocesado completo (bias field correction, z-score, resize).
- Detección de regiones sospechosas mediante ensemble de modelos binarios.
- Segmentación multiclase con ensemble voxel-wise de dos modelos especializados.
- Reconstrucción del volumen segmentado.
- Visualización avanzada multicanal y métricas volumétricas.

Además, se ofrecen funcionalidades interactivas para el usuario:

- Exploración de cortes en los tres planos anatómicos (axial, coronal y sagital).
- Selección de modalidad de imagen (T1n, T1c, T2f, T2w).
- Visualización opcional de las clases tumorales segmentadas (1–4).

- Activación de Test-Time Augmentation (TTA) para mejorar la robustez en la inferencia.
- Control del stride aplicado durante la detección.
- Descarga directa de las segmentaciones en formato .nii.gz.

Este prototipo demuestra la viabilidad de la productivización del sistema, facilitando su integración en flujos de trabajo clínicos y mejorando la interpretabilidad de los resultados para el usuario final.

Para ver una explicación breve e imágenes acceder al Anexo I, sección 8. En el anexo se incluye un enlace a un video en YouTube de la demo del programa.

7.6 Retos técnicos y soluciones

A lo largo del desarrollo del proyecto surgieron múltiples dificultades técnicas que requirieron ajustes y rediseños en el pipeline. Lejos de ser un obstáculo, estos retos constituyeron un aprendizaje clave y condicionaron varias de las decisiones metodológicas adoptadas. Entre los más relevantes destacan:

- **Inestabilidad en aumentaciones (loss: nan):**

Durante las primeras pruebas, la inclusión de rotaciones y flips aleatorios en 3D producía pérdidas nan. Tras analizar el problema, se sustituyeron por aumentaciones definidas íntegramente en TensorFlow, con chequeos de consistencia para garantizar que no se perdieran clases durante la transformación.

- **Problemas de memoria con tf.py_function:**

Al construir el tf.data.Dataset con tf.py_function, el kernel se reiniciaba por consumo excesivo de RAM. La solución fue volver a tf.numpy_function con set_shape, lo que permitió mantener formas conocidas y evitar cargas completas en memoria.

- **Desbalance extremo de clases:**

La clase 4 (cavidad de resección) resultó particularmente difícil de segmentar debido a su baja frecuencia (~12.6% de los casos). Para abordar este reto se aplicaron estrategias de oversampling de parches minoritarios y una Focal Loss con mayor peso en la clase 4, lo que mejoró la sensibilidad en este subgrupo.

- **Ensembles iniciales fallidos:**

Una primera estrategia de ensemble dirigida por clase (U-Net++ para clases 1–3 y Attention U-Net para la clase 4) empeoró las métricas globales y dio Dice nulo en la clase 4. Este resultado motivó la adopción del softmax voxel-wise ensemble, que demostró ser mucho más estable y efectivo.

- **Variabilidad entre espacios (SRI24 vs UCSD):**

Se observó que los modelos entrenados en volúmenes normalizados (SRI24) perdían rendimiento al aplicarse en el espacio nativo (UCSD). Esto puso de manifiesto la necesidad de entrenar con aumentaciones más realistas y de evaluar la robustez del pipeline frente a datos clínicos heterogéneos.

- **Entrenamiento con volúmenes completos (128³):**

En una fase inicial se intentó entrenar con volúmenes enteros de 128³, pero la arquitectura diseñada para parches 64³ no era compatible. Este hallazgo derivó en la definición de una estrategia híbrida con parches de 64³ y 96³, equilibrando detalle local y contexto global.

- **Detector binario: trade-off entre recall y AUC:**

El modelo detector binario mostraba un conflicto entre optimizar el recall (para no perder parches positivos) y el AUC (para mejorar discriminación). La solución fue entrenar dos modelos especializados y combinarlos en un ensemble de detectores, priorizando recall en la cascada.

- **Corrupción de archivos preprocesados (.npy):**

En un momento del desarrollo se detectaron inconsistencias en algunos volúmenes guardados en formato .npy. Esto llevó a reprocesar directamente desde los .nii.gz originales, garantizando integridad en los datos y homogeneidad en todo el pipeline.

En conjunto, estos retos reflejan la complejidad inherente al trabajo con datos biomédicos 3D y la importancia de un proceso iterativo de prueba, error y ajuste. Cada corrección contribuyó a robustecer el pipeline final y a reforzar la validez de los resultados obtenidos.

8. Conclusiones y Futuras Mejoras

8.1 Conclusiones principales

El presente Trabajo Fin de Máster ha abordado el problema de la detección y segmentación automática de metástasis cerebrales en el marco del reto BraTS 2025 (Task 4), desarrollando un pipeline basado en Deep Learning 3D con los siguientes logros principales:

- Se implementó un pipeline en dos etapas (detector binario + segmentador multiclase) que permitió mejorar la sensibilidad global y optimizar el uso de recursos computacionales.
- Se exploraron distintas arquitecturas de segmentación 3D (Attention U-Net, U-Net++ y variantes EfficientNet-like), adaptadas mediante deep supervision y mecanismos de atención para mejorar la precisión voxel a voxel.

- Se diseñó una función de pérdida combinada (Dice + Focal Loss con α ponderado por clase) que resultó efectiva frente al desbalance de clases, especialmente en la clase 4.
- La aplicación de estrategias de ensemble, en particular el softmax voxel-wise, mejoró de forma significativa los resultados en las clases minoritarias y demostró el valor de integrar arquitecturas complementarias.
- Se consiguió poner en producción el modelo en una aplicación interactiva, que permite cargar resonancias, obtener segmentaciones automáticas y explorar los resultados de forma intuitiva mediante selección de slices, modalidades y clases tumorales.

Además, se incluyeron ejemplos visuales de tres casos representativos de la partición de validación interna: uno con presencia de todas las clases, otro con buen rendimiento en las clases 1–3 y un tercero con dificultades en dichas clases.

Estos ejemplos ilustraron la capacidad del modelo para segmentar correctamente regiones complejas y, al mismo tiempo, pusieron de relieve los retos pendientes en clases minoritarias como la cavidad de resección.

Esta combinación de análisis cuantitativo y cualitativo refuerza la solidez de las conclusiones del presente trabajo.

Las evaluaciones cualitativas se realizaron sobre la partición de validación interna, que no fue utilizada para entrenar el modelo, por lo que representan una medida independiente del rendimiento alcanzado.

En conjunto, el trabajo confirma que el uso de redes neuronales convolucionales 3D con enfoques híbridos y ensembles es una estrategia prometedora para la segmentación de metástasis cerebrales, y que su implementación práctica en una interfaz interactiva constituye un paso relevante hacia su futura integración en el flujo clínico.

8.2 Limitaciones

A pesar de los avances logrados, el estudio presenta varias limitaciones:

- La clase 4 (cavidad de resección) continúa mostrando un rendimiento bajo debido a su escasa representación y gran variabilidad anatómica.
- El sistema puede fallar en la detección de lesiones muy pequeñas, con impacto clínico relevante en pacientes con múltiples metástasis.
- La validación se ha realizado mayoritariamente en el espacio SRI24, lo que limita la extrapolación inmediata a datos en espacio nativo y, por extensión, a entornos clínicos reales.
- El entrenamiento en parches, aunque eficiente, puede perder contexto global, dificultando la segmentación de regiones extensas o distribuidas.

8.3 Futuras mejoras

Con base en los hallazgos y limitaciones, se plantean diversas líneas de mejora para trabajos posteriores:

- **Modelos expertos por clase:** entrenar modelos dedicados exclusivamente a clases minoritarias (ej. cavidad de resección), combinados mediante ensembles inteligentes.
- **Transformers 3D y arquitecturas híbridas CNN–Transformer:** explorar nuevas arquitecturas capaces de capturar dependencias de largo alcance entre voxeles.
- **Aprendizaje auto-supervisado** (self-supervised learning): aprovechar preentrenamiento en grandes volúmenes de MRI no anotados para mejorar la robustez del modelo.
- **Generalización a espacios nativos:** incluir técnicas de normalización espacial adaptativa y aumentaciones más realistas para acercar el pipeline al entorno clínico real.
- **Extensión del prototipo productivo:** evolucionar la aplicación interactiva actual hacia un entorno clínico simulado, con integración de informes automáticos de volumetría y exportación de resultados en formato estándar (DICOM/NIfTI).
- **Segmentar nuevas secuencias de RM:** en los últimos años se está utilizando cada vez en más centros las secuencias de sangre negra con contraste (black blood MRI) para la detección de micrometástasis por su superioridad frente a las secuencias convencionales de T1 postcontraste.

9. Referencias

- Bakas, S., Reyes, M., Jakab, A., Bauer, S., Rempfler, M., Crimi, A., ... & Menze, B. H. (2018). Identifying the best machine learning algorithms for brain tumor segmentation, progression assessment, and overall survival prediction in the BRATS challenge. arXiv preprint arXiv:1811.02629.
- Isensee, F., Jaeger, P. F., Kohl, S. A., Petersen, J., & Maier-Hein, K. H. (2021). nnU-Net: a self-adapting framework for U-Net-based medical image segmentation. *Nature Methods*, 18(2), 203-211.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (pp. 234–241). Springer.
- Oktay, O., Schlemper, J., Folgoc, L. L., Lee, M., Heinrich, M., Misawa, K., ... & Rueckert, D. (2018). Attention U-Net: Learning where to look for the pancreas. arXiv preprint arXiv:1804.03999.
- Zhou, Z., Siddiquee, M. M. R., Tajbakhsh, N., & Liang, J. (2018). UNet++: Redesigning skip connections to exploit multiscale features in image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (pp. 3–11). Springer.
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 2980–2988).
- Chen, L. C., Papandreou, G., Kokkinos, I., Murphy, K., & Yuille, A. L. (2017). Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4), 834–848.

- Menze, B. H., Jakab, A., Bauer, S., Kalpathy-Cramer, J., Farahani, K., Kirby, J., ... & van Leemput, K. (2015). The multimodal brain tumor image segmentation benchmark (BRATS). *IEEE Transactions on Medical Imaging*, 34(10), 1993–2024.
- Shashwath, E. P., et al. (2023). Brain metastasis segmentation and analysis (BMSA) challenge: overview and results. In *Proceedings of MICCAI Brain Lesion Workshop*.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

ANEXOS

En estos anexos se incluyen fragmentos representativos de código (snippets), abreviados y comentados, junto con figuras explicativas. El código completo, con todas las celdas ejecutables, detalles y visualizaciones, está disponible en el Notebook HTML. Cuando procede, se han simplificado rutas e hiperparámetros; para reproducibilidad consulte el notebook.

Anexo A — Sección 1 del html: Preprocesado de datos.

Objetivo. Unificar cada caso a un tensor **128×128×128×4** aplicando N4 en T1n/T1c, normalización z-score por canal y redimensionado, manteniendo coherencia con la máscara.

Alcance. Conjuntos Training/Validation (SRI24 y UCSD). Código completo y procesamiento por lotes en el notebook.

I/O. - Entrada: t1n, t1c, t2f, t2w (.nii.gz) + seg.nii.gz si existe. - **Salida:** *_image.npy → **128×128×128×4** (float32), *_mask.npy → **128×128×128** (uint8).

Secuencia. (1) N4 en T1n/T1c (con máscara si está). (2) z-score por canal. (3) Resize 3D (imagen: orden=1; máscara: orden=0). (4) Stack de 4 canales. (5) Persistencia .npy con verificación.

Parámetros clave. Tamaño destino=128; iteraciones N4=(50,50,50,30); orden de canales: t1n,t1c,t2f,t2w; modalidad ausente→canal de ceros del tamaño de referencia.

QC. Shapes/dtypes esperados; medias≈0 y $\sigma \approx 1$ por canal (± 0.1); chequeo de alineación (overlay máscara–T1c); reprocesado automático si .npy no es legible.

Código A.1 — Preprocesado (N4 + z-score + resize a 128³)

```
import numpy as np, SimpleITK as sitk
from scipy.ndimage import zoom

def n4_bias_correct(vol, mask=None, its=(50,50,50,30)):
    img = sitk.GetImageFromArray(vol.astype(np.float32))
    msk = sitk.GetImageFromArray(mask.astype(np.uint8)) if mask is not None else None
    corr = sitk.N4BiasFieldCorrectionImageFilter()
    corr.SetMaximumNumberOfIterations(list(its))
    out = corr.Execute(img, msk) if msk else corr.Execute(img)
    return sitk.GetArrayFromImage(out)

def zscore_per_channel(x, eps=1e-5):
    x = x.copy()
    for c in range(x.shape[-1]):
        vals = x[:, :, :, c]
        vals_nz = vals[vals != 0]
        m = vals_nz.mean() if vals_nz.size else 0.0
```

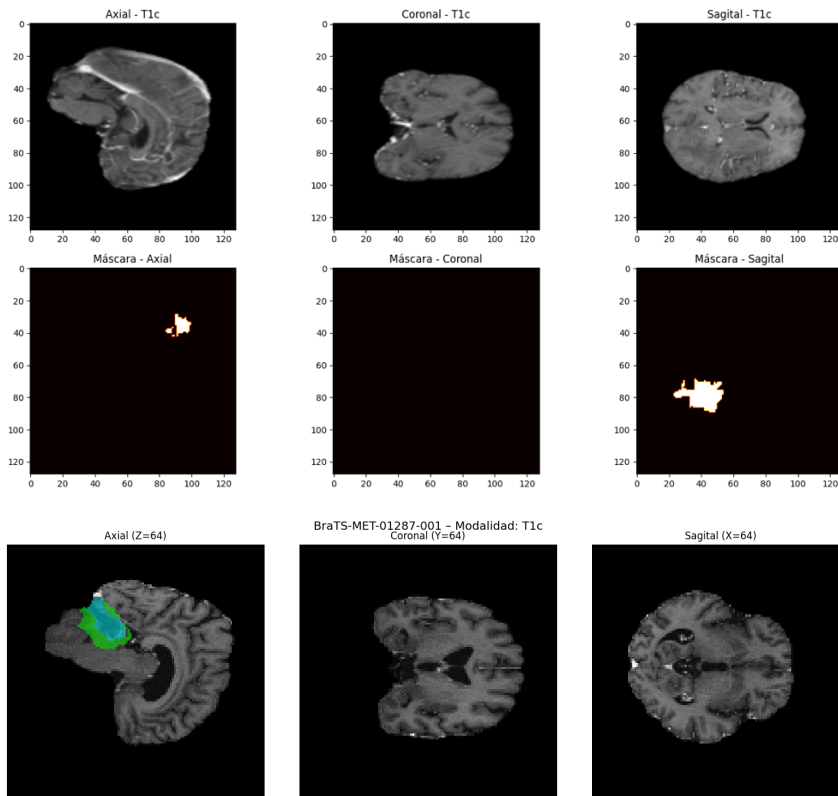
```

s = vals_nz.std() if vals_nz.size else 1.0
x[..., c] = (vals - m) / max(s, eps)
return x

def preprocess_modalities(t1n, t1c, t2f, t2w, seg=None, target=128):
    t1n = n4_bias_correct(t1n, seg>0 if seg is not None else None)
    t1c = n4_bias_correct(t1c, seg>0 if seg is not None else None)
    def rz(v, order):
        zf = (target / v.shape[0], target / v.shape[1], target / v.shape[2])
        return zoom(v, zf, order=order)
    img4 = np.stack([rz(t1n,1), rz(t1c,1), rz(t2f,1), rz(t2w,1)], axis=-1).astype(np.float32)
    segR = rz(seg,0).astype(np.uint8) if seg is not None else None
    return zscore_per_channel(img4), segR # -> (128,128,128,4), (128,128,128)

```

Visualización de imágenes preprocesadas.



Anexo B — Sección 3 del html: Creación de Patches

Objetivo. Crear parches 3D (64^3 y 96^3) equilibrando la representación de clases (oversampling) y estructurar un tf.data eficiente.

Alcance. Conjuntos Training/Validation; split por **paciente** para evitar fuga de información.

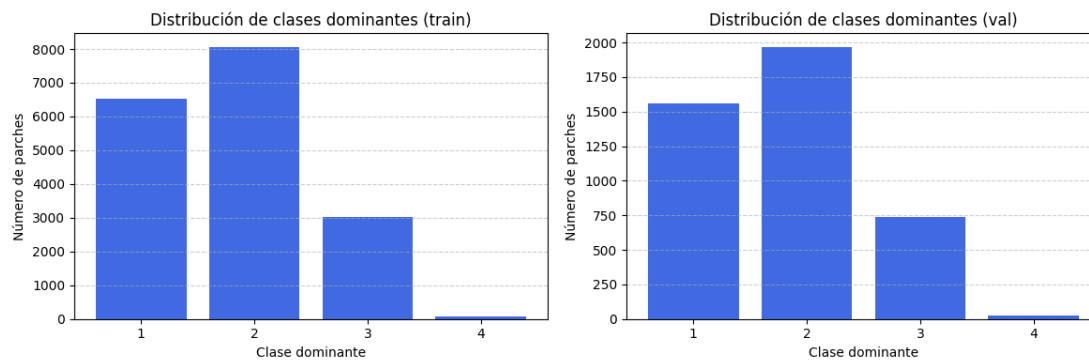
I/O. - Entrada: volúmenes preprocesados ($128 \times 128 \times 128 \times 4$) y máscaras ($128 \times 128 \times 128$).
- **Salida:** carpetas patches_64/ y patches_96/ separadas en train/ y val/; índices por paciente.

Secuencia. (1) Indexado de voxels positivos por clase. (2) Muestreo de centros con **oversampling** (más para clases 1 y 4). (3) Extracción de parches $64^3/96^3$ (con negativos). (4) Guardado en disco. (5) Pipeline tf.data con map augment → batch → prefetch.

Parámetros clave. Tamaños de parche (64 y 96); ratios de oversampling por clase (p. ej., clase 4 > clase 1 > resto); seed fija; mezcla estratificada por paciente.

QC. Distribución por clase (tabla/plot); verificación de shapes y dtypes; muestreo visual de parches positivos/negativos.

Distribución de Clases Dominantes.



Código B.1 — Muestreo y extracción de parches

```
import numpy as np, random

def sample_centers(mask, k_per_class, neg_ratio=0.33):
    pos = {c: np.argwhere(mask == c) for c in [1,2,3,4]}
    centers = []
    for c, k in k_per_class.items():
        pts = pos.get(c, [])
        if len(pts) == 0: continue
        sel = pts[np.random.choice(len(pts), size=min(k, len(pts)), replace=True)]
        centers.extend([(tuple(p), c) for p in sel])
    neg_pool = np.argwhere(mask == 0)
    n_neg = int(sum(k_per_class.values()) * neg_ratio)
    for _ in range(min(n_neg, len(neg_pool))):
        centers.append((tuple(neg_pool[random.randrange(len(neg_pool))]), 0))
```

```

return centers # lista de ((z,y,x), clase)

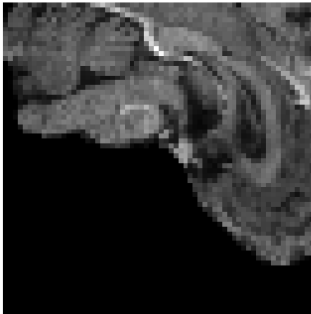
def extract_patch(vol4, seg, center, size):
    z,y,x = center; h = size // 2
    z0,z1 = max(0,z-h), min(vol4.shape[0], z+h)
    y0,y1 = max(0,y-h), min(vol4.shape[1], y+h)
    x0,x1 = max(0,x-h), min(vol4.shape[2], x+h)
    return vol4[z0:z1, y0:y1, x0:x1, :], seg[z0:z1, y0:y1, x0:x1]

```

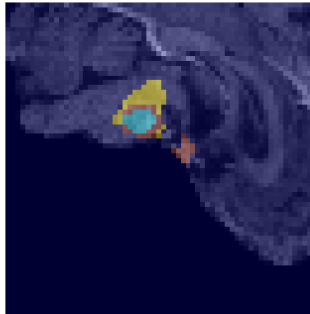
Visualizacion de parche.

BraTS-MET-01210-000_32_32_32.npz | label=1 | case=BraTS-MET-01210-000

T1c (canal 1)



Máscara superpuesta



Anexo C — Sección 4 del html: Modelo Detector Binario (etapa 1)

Objetivo. Maximizar **recall/AUC** para localizar regiones candidatas a metástasis, reduciendo el espacio para la segmentación.

Alcance. Parches 64^3 ; etiquetas binarias (tumor vs. no tumor). Se evalúan dos variantes (optimizada a AUC y a recall) para **ensemble** posterior.

I/O. - **Entrada:** parches $64^3 \times 4$ y etiqueta binaria por parche. - **Salida:** probabilidades por parche; umbrales operativos (p. ej., $t=0.35/0.5$).

Secuencia. (1) Entrenamiento con pérdida binaria (BCE/Focal). (2) Calibración de umbral según **ROC/PR**. (3) Exportación de mejores checkpoints.

Parámetros clave. batch_size, lr con warm-up, early_stopping, augmentaciones ligeras; criterio de selección (val_AUC o val_recall).

QC. Curvas ROC/PR; matriz de confusión; análisis de **recall** por paciente; verificación de coordenadas reconstruidas.

Código C.1 — Arquitectura y compilación del detector

```
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers

def conv_block(x, f):
    s = x
    x = layers.Conv3D(f,3,padding="same",kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.BatchNormalization()(x); x = layers.LeakyReLU(0.1)(x)
    x = layers.Conv3D(f,3,padding="same",kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.BatchNormalization()(x)
    if s.shape[-1] == f:
        x = layers.Add()([x, s])
    x = layers.LeakyReLU(0.1)(x); x = layers.MaxPool3D(2)(x)
    return x

def build_detector(input_shape=(64,64,64,4)):
    inp = layers.Input(input_shape)
    x = conv_block(inp,32); x = conv_block(x,64); x = conv_block(x,128); x =
conv_block(x,256)
    x = layers.GlobalAveragePooling3D()(x)
    x = layers.Dense(64, activation="relu")(x)
    out = layers.Dense(1, activation="sigmoid", dtype="float32")(x)
    return models.Model(inp, out, name="Detector3D")

def focal_loss(gamma=2., alpha=0.25):
    def _f(y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
```

```
bce = tf.keras.backend.binary_crossentropy(y_true, y_pred)
return alpha * tf.pow(1 - tf.exp(-bce), gamma) * bce
return _f

model = build_detector()
model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
              loss=focal_loss(2.,0.25),
              metrics=[tf.keras.metrics.AUC(name="auc"),
                      tf.keras.metrics.Recall(name="recall")])
```

Anexo D — Sección 5 del html: Segmentadores 3D (etapa 2)

Objetivo. Segmentar multiclase (0–4) con dos modelos complementarios: **UNet++ 96³** (clases 1–3) y **Attention UNet 64³** (clase 4 y lesiones pequeñas).

Alcance. Parches 64³ y 96³; salida softmax 5 canales; supervisión profunda (deep supervision) cuando aplica.

I/O. - Entrada: parches multiclase con máscara correspondiente. - **Salida:** logits/softmax por voxel (formato .npz/.npy) y máscaras reconstruidas por volumen.

Secuencia. (1) Definición de arquitecturas y cabezas softmax. (2) Entrenamiento con **Dice+Focal** (ponderada por clase). (3) Reconstrucción a volumen completo; opcional **TTA**.

Parámetros clave. Activación final softmax (5 clases, include_bg=False si aplica); alpha Focal (p. ej., (0.2,1,1,1,3)); mixed_precision; Conv3DTranspose vs upsampling; LeakyReLU, dropout.

QC. Dice por clase (1–4); curvas de aprendizaje; inspección visual en 3 planos; control de NaNs y estabilidad de pérdidas.

Código D.1 — Esqueleto U-Net++ 3D con Deep Supervision

```
import tensorflow as tf
from tensorflow.keras import layers, models

def unetpp_block(x, skips, f):
    x = layers.Conv3D(f,3,padding="same")(x); x = layers.LeakyReLU(0.1)(x)
    for s in skips: x = layers.Concatenate()([x, s])
    x = layers.Conv3D(f,3,padding="same")(x); x = layers.LeakyReLU(0.1)(x)
    return x

def unetpp_3d(input_shape=(96,96,96,4), classes=5, deep_supervision=True):
    inp = layers.Input(input_shape)
    e1 = layers.Conv3D(32,3,padding="same",activation="relu")(inp); d1 = layers.MaxPool3D(2)(e1)
    e2 = layers.Conv3D(64,3,padding="same",activation="relu")(d1); d2 = layers.MaxPool3D(2)(e2)
    e3 = layers.Conv3D(128,3,padding="same",activation="relu")(d2); d3 = layers.MaxPool3D(2)(e3)
    b = layers.Conv3D(256,3,padding="same",activation="relu")(d3)
    u3 = layers.UpSampling3D()(b); d3p = unetpp_block(u3, [e3], 128)
    u2 = layers.UpSampling3D()(d3p); d2p = unetpp_block(u2, [e2, layers.UpSampling3D()(e3)], 64)
    u1 = layers.UpSampling3D()(d2p); d1p = unetpp_block(u1, [e1, layers.UpSampling3D()(e2), layers.UpSampling3D()(e3)], 32)
    out_main = layers.Conv3D(classes,1,activation="softmax", name="out_main")(d1p)
```

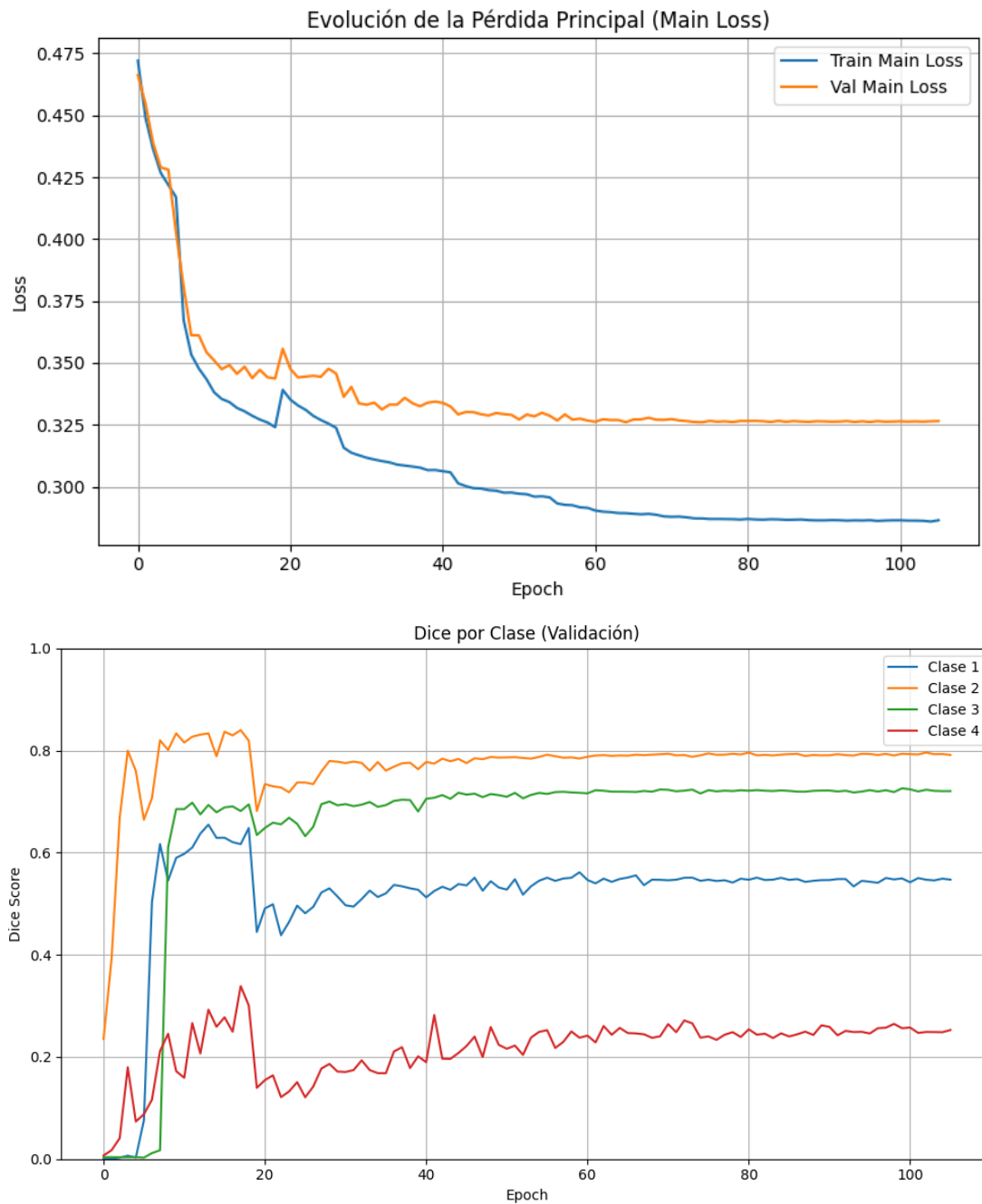
```

if deep_supervision:
    out_d2 = layers.Conv3D(classes,1,activation="softmax", name="out_d2")(d2p)
    out_d3 = layers.Conv3D(classes,1,activation="softmax", name="out_d3")(d3p)
    return models.Model(inp, [out_main, out_d2, out_d3])
return models.Model(inp, out_main)

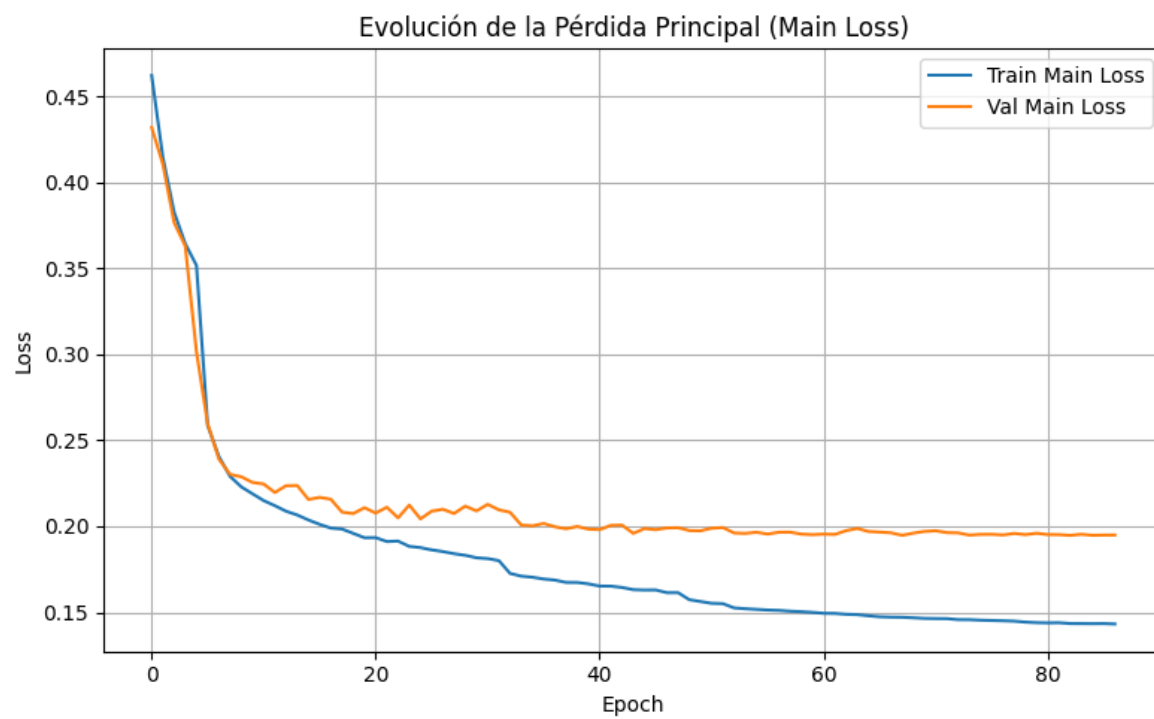
```

Evolución del Entrenamiento del modelo usado en inferencia.

Modelo de 64:



Modelo de 96:



Anexo E — Sección 5 del html: Entrenamiento y pérdidas

Objetivo. Consolidar hiperparámetros, política de LR y **función de pérdida combinada** para clases desbalanceadas (énfasis en clase 4).

Alcance. Aplica a detector y segmentadores; contempla **deep supervision**, warm-up y ReduceLROnPlateau.

I/O. - Entrada: datasets estratificados (train/val) y configuración de entrenamiento. -

Salida: mejores checkpoints, logs (CSVLogger/TensorBoard).

Secuencia. (1) Warm-up → base LR. (2) ReduceLROnPlateau por val_out_main_loss. (3) EarlyStopping con restauración de pesos. (4) Guardado de mejor/último.

Parámetros clave. Combinación **Dice + Focal ($\gamma \approx 1.5$; α por clase)**; batch_size según memoria; TTA (en inferencia); seed fija.

QC. Convergencia estable; ausencia de NaNs; gap train/val razonable; revisión de logs y checkpoints recuperables.

Código E.1 — Pérdidas combinadas y *annealing* de DS

```
import tensorflow as tf

def dice_loss(y_true, y_pred, smooth=1e-5):
    y_true = tf.cast(y_true, tf.float32)
    num = 2*tf.reduce_sum(y_true*y_pred, axis=[1,2,3,4])
    den = tf.reduce_sum(y_true+y_pred, axis=[1,2,3,4]) + smooth
    return 1 - tf.reduce_mean(num/den)

def focal_mc(y_true, y_pred, gamma=1.5, alpha=(0.2,1,1,1,3)):
    y_true = tf.cast(y_true, tf.float32); alpha = tf.constant(alpha, dtype=tf.float32)
    ce = -tf.reduce_sum(alpha * y_true * tf.math.log(y_pred+1e-7) * tf.pow(1-y_pred,
gamma), axis=-1)
    return tf.reduce_mean(ce)

def combined_loss(y_true, y_pred):
    return dice_loss(y_true, y_pred) + focal_mc(y_true, y_pred)

def ds_weights(epoch, T=50, w0=(1.0, 0.5, 0.25)):
    t = min(epoch, T)/T
    return [w0[0], w0[1]*(1-t), w0[2]*(1-t**2)]

class WarmupScheduler(tf.keras.callbacks.Callback):
    def __init__(self, base_lr=1e-4, warm_epochs=3):
        self.base_lr=base_lr; self.w=warm_epochs
    def on_epoch_begin(self, epoch, logs=None):
```



```
lr = self.base_lr * min(1., (epoch+1)/self.w)  
tf.keras.backend.set_value(self.model.optimizer.lr, lr)
```

Anexo F — Sección 6 del html: Ensemble voxel-wise por clase (con TTA)

Objetivo. Combinar softmax de modelos complementarios **por clase y por voxel**, con agregación TTA para robustez.

Alcance. Clases 1–3 desde UNet++ 96³; clase 4 desde Attention UNet 64³; fondo (clase 0): promedio.

I/O. - Entrada: volúmenes softmax (con y sin TTA). - **Salida:** máscara final multiclase y métricas por clase.

Secuencia. (1) Inferencia con TTA (flips 3D). (2) Promedio de probabilidades por TTA. (3) Selección por clase/voxel. (4) Argmax final → máscara.

Parámetros clave. Lista de TTA (flipX/Y/Z); normalización de probabilidades; control de empates; umbral opcional para clase 4.

QC. Mejora de Dice por clase vs. modelos individuales; análisis de casos fallidos; estabilidad en bordes.

Código F.1 — TTA por flips y combinación por clase

```
import numpy as np

def tta_flips(vol):
    outs = []
    axes = [(), (0,), (1,), (2,), (0,1), (0,2), (1,2), (0,1,2)]
    for ax in axes:
        v = np.flip(vol, axis=ax) if ax else vol
        inv = (lambda x, ax=ax: np.flip(x, axis=ax)) if ax else (lambda x, ax=ax: x)
        outs.append((v, inv))
    return outs

def ensemble_softmax(soft64, soft96):
    out = np.zeros_like(soft64)
    out[...,0] = 0.5*(soft64[...,0] + soft96[...,0]) # fondo
    out[...,1:4] = soft96[...,1:4] # clases 1–3
    out[...,4] = soft64[...,4] # clase 4
    return out
```

Modelos con y sin TTA.



Anexo G — Sección 6.5 del html: Evaluación y métricas

Objetivo. Evaluar por caso y por clase usando **Dice**, **NSD** y **HD95**, y cuantificar **volumetría** (GT vs. predicción).

Alcance. Conjunto de validación oficial y/o validación interna.

I/O. - Entrada: máscaras GT y predichas; (opcional) softmax para análisis umbralado. -

Salida: tablas por clase/caso; resúmenes (media/mediana/IQR); gráficos complementarios.

Secuencia. (1) Cálculo por clase/caso. (2) Agregación (macro/por clase). (3) Volúmenes (mm³/voxels). (4) Visuales (box/violin) y overlays.

Parámetros clave. Definición de superficie para NSD; voxel spacing si aplica; manejo de casos sin clase presente.

QC. Sanity checks (Dice=1 cuando GT=pred); consistencia de volumetría; revisión de outliers.

Código G.1 — Cálculo simple de Dice por clase

```
import numpy as np

def dice_per_class(y_true, y_pred, classes=(1,2,3,4), eps=1e-7):
    out = {}
    for c in classes:
        t = (y_true==c); p = (y_pred==c)
        num = 2*np.logical_and(t,p).sum()
        den = t.sum() + p.sum() + eps
        out[c] = num/den
    return out # {clase: dice}

# NSD/HD95: usar librerías de superficie (p. ej., 'surface-distance').
```

Resultados.

Modelo 64:

Clase	Dice (mediana)	HD95 (media)	NSD (mediana)
1 – NETC	0.7586	7.15 mm	0.9167
2 – SNFH	0.7557	7.41 mm	0.8813
3 – ET	0.7610	9.11 mm	0.9351
4 – RC	0.6187	6.63 mm	0.6667

Modelo 96:

Clase	Dice (mediana)	HD95 (media)	NSD (mediana)
1 – NETC	0.8000	5.52 mm	0.8333
2 – SNFH	0.8010	6.72 mm	0.8836
3 – ET	0.7500	9.46 mm	0.9091
4 – RC	0.0000	13.18 mm	0.4906

Anexo H — Sección 7 del html: Estudio de casos

Objetivo. Ilustrar el funcionamiento del pipeline **detección** → **segmentación (ensemble voxel-wise + TTA)** y valorar su rendimiento **cualitativamente** en escenarios clínicos variados.

Alcance. Tres casos de la **validación** seleccionados por representatividad clínica (todas las clases, casos sin RC, y caso difícil).

Casos.

- **BraTS-MET-01071-001:** presencia de **NETC (1)**, **SNFH (2)**, **ET (3)** y **RC (4)**.
- **BraTS-MET-01087-001:** buen desempeño en **1–2–3**, **sin** cavidad de resección (4).
- **BraTS-MET-00286-000:** **caso difícil**, útil para analizar límites del modelo.

I/O.

- **Entrada:** volumen **128×128×128×4** (T1n, T1c, T2 FLAIR, T2) preprocesado.
- **Salida:** **softmax** promedio por voxel (5 canales) y **máscara final** (argmax).

Secuencia.

1. **Corte volumétrico** 128^3 con 4 modalidades.
2. **Detección (ensemble binario)** orientada a **recall** (umbral por PR/F2) → selección de parches relevantes.
3. **Segmentación voxel-wise** con **UNet++ 64^3** y **UNet++ 96^3** , **TTA** (flips 3D) y **ensemble por clase**:
 - Clases **1–2–3** → modelo **96^3** .
 - Clase **4** → modelo **64^3** .
 - **Fondo (0)** → promedio de ambos.
4. **Reconstrucción volumétrica** y **argmax** del softmax promedio.
5. **Visualización** y anotación cualitativa.

Parámetros clave.

- **Patch sizes:** 64^3 y 96^3 ; **stride** adaptado al caso.
- **Detector:** umbral t^* elegido por **PR/F2** (prioriza sensibilidad).

- **TTA:** flips 3D; agregación por **promedio** en probabilidades.
- **Colores:** 1-NETC **rojo**, 2-SNFH **verde**, 3-ET **amarillo**, 4-RC **cian**.

Interpretación esperada.

- El **ensemble voxel-wise** es **muy fiable** en **estructuras amplias y definidas** (clases **1–2–3**).
- **Clase 4 (RC)** mejora respecto a modelos individuales, aunque sigue siendo la **más retadora** (formas irregulares).
- En **casos atípicos o con baja SNR**, pueden aparecer **fallos estructurales**; el pipeline en **cascada** y el **TTA** ayudan a **contener FP** y estabilizar **bordes**.

QC.

- Coherencia **imagen–máscara** (alineación visual).
- Ausencia de “**sangrados**” de clase entre tejidos y cavidad.
- Revisión de **outliers** (FP aislados, FN en lesiones pequeñas)

Resultados (resumen).

- Los tres casos muestran **mejoras con TTA** en la **definición de borde** y la **consistencia** entre slices.
- En el caso **01071-001** se observa recuperación de **clase 4** y buen acuerdo en **1–2–3**.
- En **01087-001** la ausencia de RC se refleja como **cero** en clase 4, manteniendo buen desempeño en **1–2–3**.
- En **00286-000** persisten **zonas difíciles**, útiles para orientar **futuros post-procesos** (p. ej., **CRF** o reglas morfológicas).

Código H.1 — Búsqueda de slices y overlay coloreado

```
import numpy as np

def best_axial_slices_by_class(mask, classes=(1,2,3,4)):
    best = {}
    for c in classes:
        counts = [(z, (mask[z]==c).sum()) for z in range(mask.shape[0])]
        best[c] = max(counts, key=lambda x:x[1])
    return best # {clase: (z, n_vox)}

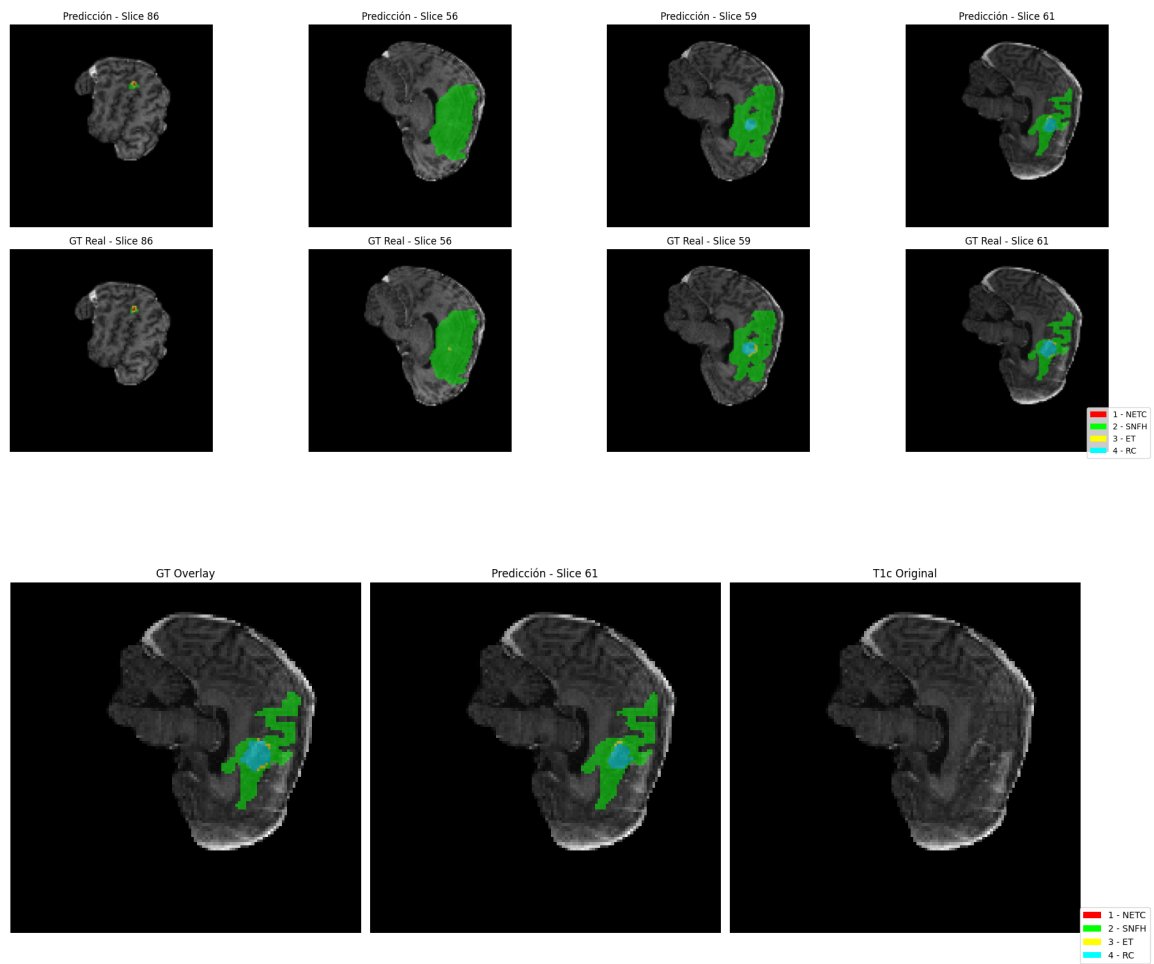
def make_overlay_gray(img2d, seg2d, cmap={1:(1,0,0),2:(0,1,0),3:(1,1,0),4:(0,1,1)},
alpha=0.5):
    img = (img2d - img2d.min())/(img2d.max()-img2d.min()+1e-7)
```

```

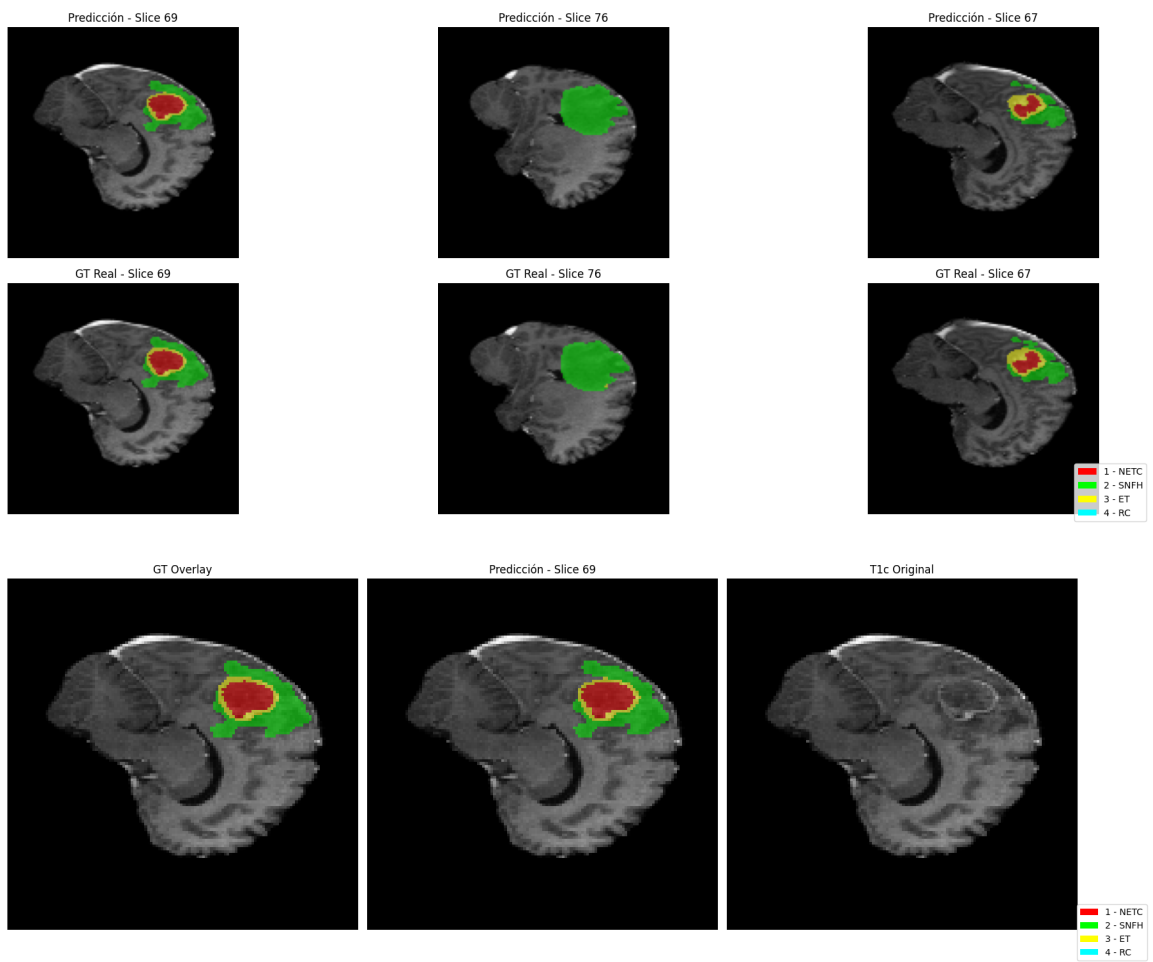
rgb = np.stack([img,img,img], -1)
for c, col in cmap.items():
    m = (seg2d==c)[...,None]
    rgb = np.where(m, (1-alpha)*rgb + alpha*np.array(col), rgb)
return rgb

```

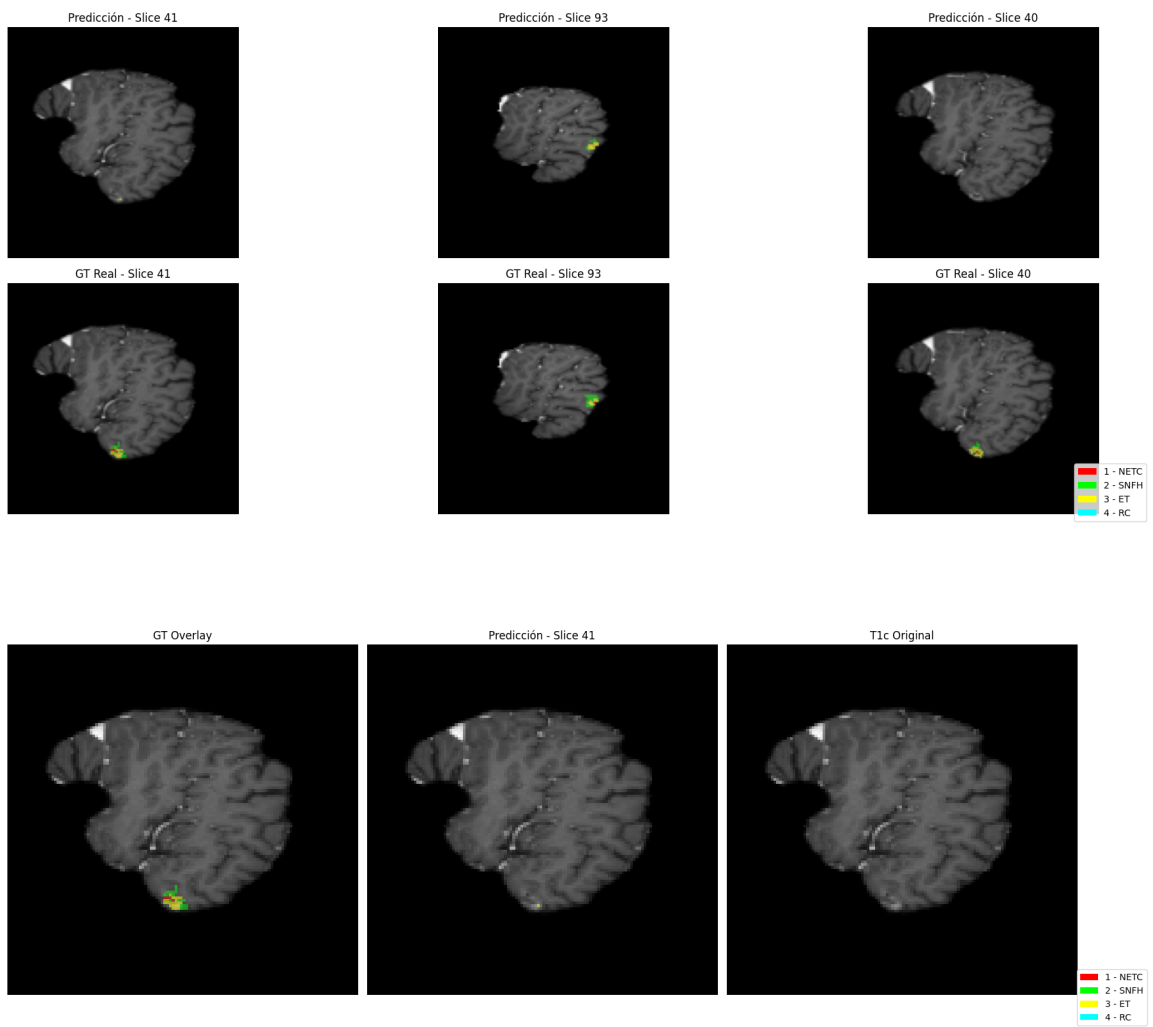
BraTS-MET-01071-001



BraTS-MET-01087-001



BraTS-MET-00286-000



Anexo I — Sección 8 del html: Productivización en Streamlit

Objetivo. Permitir carga de .npy o 4 .nii.gz, preprocesar y ejecutar el pipeline **detección + segmentación + visualización** con opciones de TTA y descarga.

Alcance. Demo académica (no uso clínico). Funciona localmente o en entorno GPU.

I/O. - Entrada: *_image.npy o (t1n,t1c,t2f,t2w en .nii.gz). - **Salida:** overlays, métricas de volumen, máscaras descargables (.nii.gz/.npy).

Secuencia. (1) Carga y preprocesado. (2) Detección (opcional umbral). (3) Segmentación (con/sin TTA). (4) Visualización multiplanar y descarga.

Parámetros clave. Rutas a modelos (.keras); toggle TTA; selección de clases/planos; session_state para reutilizar resultados.

QC. Comprobación de shapes; tiempos por etapa; pruebas con casos muestra.

Código I.1 — Esqueleto mínimo de la interfaz

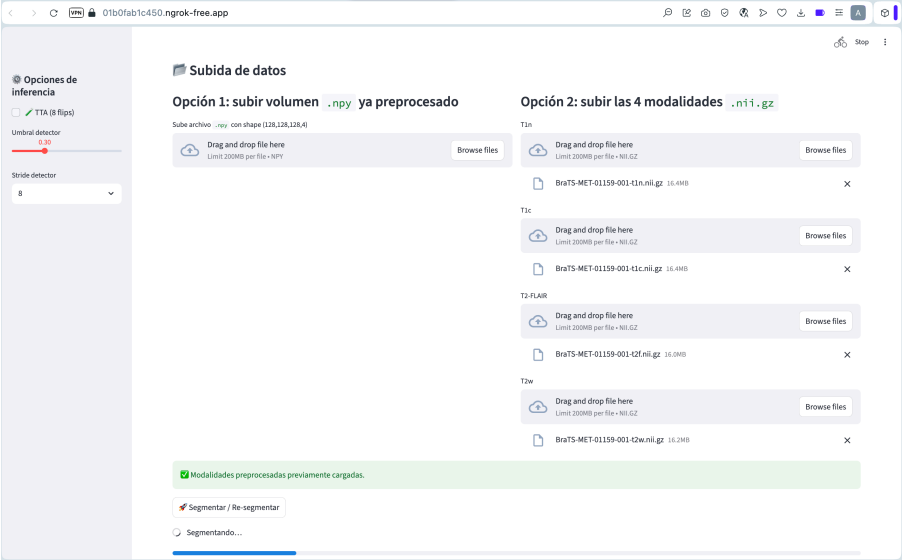
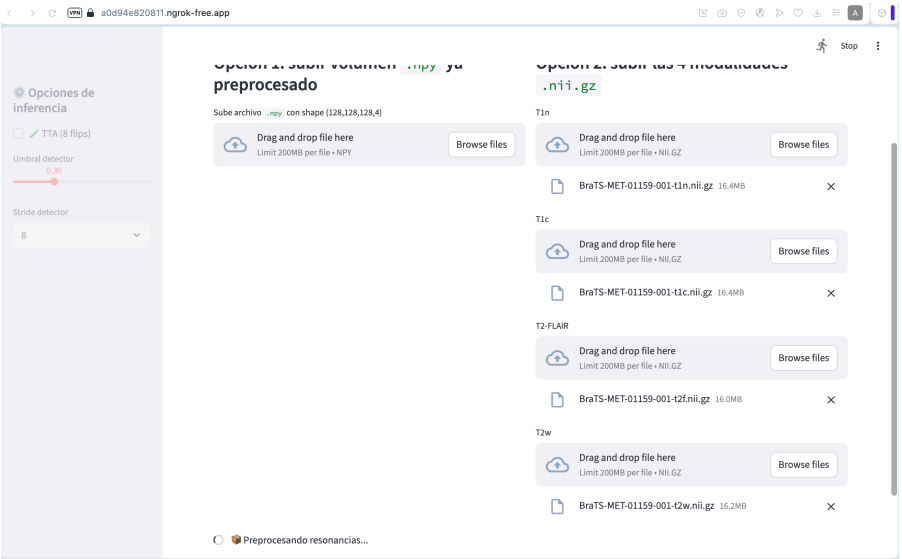
```
import streamlit as st, numpy as np

st.title("Detección + Segmentación de Metástasis (demo)")
opt = st.radio("Entrada", ["4 NIfTI (T1n,T1c,FLAIR,T2)", "Tensor .npy (128×128×128×4)"])

if opt.startswith("4 NIfTI"):
    f_t1n = st.file_uploader("T1n (.nii.gz)"); f_t1c = st.file_uploader("T1c (.nii.gz)")
    f_flr = st.file_uploader("FLAIR (.nii.gz)"); f_t2w = st.file_uploader("T2 (.nii.gz)")
else:
    f_npy = st.file_uploader(".npy (128×128×128×4)")

st.write("(Demo) Tras cargar, se aplica preprocesado → detector → segmentación (TTA) y se muestran overlays/volúmenes.")
```

Imágenes.



Opciones de inferencia

☐ TTA (8 flips)

Umbral detector

Stride detector

Segmentar / Re-segmentar

Modalidad

Vista

Slice

☒ Mostrar máscara

1 - NETC
 2 - SNFH
 3 - ET
 4 - RC

[Descargar overlay + leyenda \(.png\)](#)

Volumen por clase (voxeles $\approx$ mm³)

Opciones de inferencia

☐ TTA (8 flips)

Umbral detector

Stride detector

Segmentar / Re-segmentar

Modalidad

Vista

Slice

☐ Mostrar máscara

Volumen por clase (voxeles $\approx$ mm³)

```

{
  "Clase 1" : 2166
}
          
```



Link Youtube. Haciendo [click aqui](#) se les redigirá a un video en Youtube para visualizar un caso y su detección y segmentación en tiempo real. <https://youtu.be/LmGKAiEejPQ>

Anexo L —Acceso a los datos y modelos entrenados.

Se proporciona [este enlace](#) de google Drive para acceder a la carpeta contenedora de datos y modelos entrenados.

También puede encontrarse en [GitHub](#) el código del notebook y la app.py con la se puso en producción mediante Streamlit. https://github.com/ArgonautaSilentis/Cuaderno-Del-Argonauta/tree/main/TFM_CNN_BrainTumors_3D